



**"RANKED 45  
IN INDIA**

#University Category



**WORLD  
UNIVERSITY  
RANKINGS**

**NO.1** PVT. UNIVERSITY IN  
ACADEMIC REPUTATION IN INDIA



**ACCREDITED GRADE 'A'**  
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT  
TO EXCEPTIONAL E-LEARNING METHODS

1



# Applied Machine Learning

## Unit 04 – Lecture 05: Data Cleaning

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Missing data, outliers and transformation

Topic focus: **the model is the easy part**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.

Why clean  
oooo

Missing  
oooooooooooooooo

Shape  
ooooooo

Outliers  
oooooooooooooooo

Transform  
oooooooooooo

Order  
oooo

Summary  
oo

## Quick Links

Why clean

Missing data

Outliers

Transformation

Order of operations

Summary

# Agenda

Data preprocessing: why bother

Handling missing data

Describing the shape of a column

Handling outliers

Data transformation

The order of operations

Summary

## How this lecture works

### Every idea appears twice

1. **By hand** — eight marks, thirty-five sensor readings, nine prices. Small enough to check on paper.
2. **In code** — the same arithmetic on real data, so you see that `SimpleImputer` does exactly that and nothing more.

### Commit before you look

Four slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **a rule that cannot detect the outlier sitting in front of it**. Everything before it exists to make that failure obvious.

## Where we are

### Units I–III gave us

A model, a loss, and five ways to descend it. All of them assumed a clean numeric matrix  $X$  with no holes in it.

### Real data is not like that

Values are missing, mistyped, duplicated, in the wrong units, or simply wrong. **No optimiser on the last four slide decks can rescue any of that.**

### Unit IV starts here

Data preprocessing: cleaning (today), then feature engineering and reduction, then splitting and validation.

### Today's four questions

What do I do about a hole? About a value that looks impossible? About a variable with a long tail? **And in what order?**

## What “dirty” actually means

| Defect       | Looks like                                           | Today? |
|--------------|------------------------------------------------------|--------|
| incomplete   | a blank cell, a NaN, a $-1$ that means “not asked”   | yes    |
| noisy        | a reading that is wrong by a little                  | yes    |
| outlying     | a reading that is wrong by a lot — or right and rare | yes    |
| skewed       | one long tail, so a mean means nothing               | yes    |
| inconsistent | "M", "Male", "male", 1                               | L6     |
| duplicated   | the same patient entered twice                       | L6     |

### Han, Kamber and Pei's four tasks

**Cleaning** (today), **integration**, **reduction** and **transformation** (today and L6). Cleaning is first because everything downstream inherits its mistakes.

## Two datasets, one seed

### The data

Two bundled tables: `load_diabetes` (442 patients, 10 features, used in both scaled and raw units) and `load_breast_cancer` (569 patients, 30 features). Nothing is downloaded.

### The randomness

One seed for the whole lecture:  
`np.random.default_rng(0)`,  
`random_state=0`. Every listing that shuffles or samples shows it.

### Two deliberate exceptions

The 35 sensor readings come from `default_rng(7)`: that seed *defines the data*, so it stays. The repeated-split experiments sweep `random_state` over `range(0, 20)` or `range(0, 300)`, because the spread across splits *is* the measurement. Run the code as written and every number here comes back.

## Predict first #1

load\_breast\_cancer: 569 patients, 30 features. Knock out **5% of the individual cells**, completely at random. Then do the obvious thing — drop every row that has a hole in it.

**What fraction of the 569 patients survives?**

## Predict first #1

load\_breast\_cancer: 569 patients, 30 features. Knock out **5% of the individual cells**, completely at random. Then do the obvious thing — drop every row that has a hole in it.

**What fraction of the 569 patients survives?**

|                                          |             |
|------------------------------------------|-------------|
| fraction of individual cells missing     | 0.051       |
| rows with at least one missing value     | 455 of 569  |
| rows left after listwise deletion        | 114 (20.0%) |
| predicted survival rate $0.95 \times 30$ | $= 0.215$   |

## Predict first #1

load\_breast\_cancer: 569 patients, 30 features. Knock out **5% of the individual cells**, completely at random. Then do the obvious thing — drop every row that has a hole in it.

**What fraction of the 569 patients survives?**

|                                       |             |
|---------------------------------------|-------------|
| fraction of individual cells missing  | 0.051       |
| rows with at least one missing value  | 455 of 569  |
| rows left after listwise deletion     | 114 (20.0%) |
| predicted survival rate $0.95^{**30}$ | $= 0.215$   |

Five per cent of the cells cost you eighty per cent of the rows

A row survives only if all 30 of its cells do:  $0.95^{30} = 0.215$ . **Listwise deletion is exponential in the number of columns**, and nobody's intuition is exponential.



## Was deleting them at least safe?

|                                         |        |            |
|-----------------------------------------|--------|------------|
| accuracy on the 114 survivors           | 0.9735 | +/- 0.0216 |
| accuracy imputing instead, all 569 rows | 0.9719 | +/- 0.0129 |
| baseline with no missingness at all     | 0.9807 | +/- 0.0065 |

## Was deleting them at least safe?

|                                         |        |            |
|-----------------------------------------|--------|------------|
| accuracy on the 114 survivors           | 0.9735 | +/- 0.0216 |
| accuracy imputing instead, all 569 rows | 0.9719 | +/- 0.0129 |
| baseline with no missingness at all     | 0.9807 | +/- 0.0065 |

### Be honest about this

On *this* easy dataset deletion did not lose accuracy. It lost **455 patients** and widened the spread of the estimate by  $1.7\times$ .

### It is not always this kind

On `load_diabetes` at 30% missing, listwise deletion leaves **9 training rows** for 10 features and the fitted model scores  $R^2 = -4.70$ .

$R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$ , so  $R^2 < 0$  means *worse than always predicting  $\bar{y}$* .

## Was deleting them at least safe?

|                                         |        |            |
|-----------------------------------------|--------|------------|
| accuracy on the 114 survivors           | 0.9735 | +/- 0.0216 |
| accuracy imputing instead, all 569 rows | 0.9719 | +/- 0.0129 |
| baseline with no missingness at all     | 0.9807 | +/- 0.0065 |

### Be honest about this

On *this* easy dataset deletion did not lose accuracy. It lost **455 patients** and widened the spread of the estimate by  $1.7\times$ .

### It is not always this kind

On `load_diabetes` at 30% missing, listwise deletion leaves **9 training rows** for 10 features and the fitted model scores  $R^2 = -4.70$ .  
 $R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$ , so  $R^2 < 0$  means *worse than always predicting  $\bar{y}$* .

**Deletion is a decision to throw away evidence.** Sometimes it is the right one. It is never the free one.

## Three questions to ask about a hole

1. **Why is it missing?** A sensor failed, a question was refused, a test was never ordered — three different problems with three different fixes.

## Three questions to ask about a hole

1. **Why is it missing?** A sensor failed, a question was refused, a test was never ordered — three different problems with three different fixes.
2. **Is the reason related to the value itself?** This is the MCAR / MAR / MNAR question, and it decides whether *any* imputer can save you.

## Three questions to ask about a hole

1. **Why is it missing?** A sensor failed, a question was refused, a test was never ordered — three different problems with three different fixes.
2. **Is the reason related to the value itself?** This is the MCAR / MAR / MNAR question, and it decides whether *any* imputer can save you.
3. **Is the fact of the hole informative?** If a test is ordered only for patients who already look ill, then `isnull()` is a feature.

## Three questions to ask about a hole

1. **Why is it missing?** A sensor failed, a question was refused, a test was never ordered — three different problems with three different fixes.
2. **Is the reason related to the value itself?** This is the MCAR / MAR / MNAR question, and it decides whether *any* imputer can save you.
3. **Is the fact of the hole informative?** If a test is ordered only for patients who already look ill, then `isnull()` is a feature.

The mistake this section exists to prevent

Reaching for `fillna(df.mean())` without asking any of the three.

## By hand: eight marks, two missing

55, 62, ?, 71, 58, ?, 90, 64

Sort the six we have: 55, 58, 62, 64, 71, 90.

|                |                                             |         |
|----------------|---------------------------------------------|---------|
| mean           | $(55 + 62 + 71 + 58 + 90 + 64)/6 = 400/6$   | 66.6667 |
| median         | $(62 + 64)/2$                               | 63.0000 |
| sd ( $n - 1$ ) | $\sqrt{\frac{1}{5} \sum (x_i - \bar{x})^2}$ | 12.6754 |

Mean, median or mode?

**Mean** for a symmetric numeric column. **Median** when it is skewed — here 90 drags the mean 3.7 marks above the median. **Mode** for a categorical column, where a mean is meaningless.

## By hand: what mean imputation does to the spread

Fill both holes with  $\bar{x}$ . The mean cannot move — you added two copies of it. The *sum of squared deviations* cannot move either, because the two new deviations are zero. But you now divide it among more values:

$$\sigma_{\text{after}}^2 = \frac{\sum_{i=1}^m (x_i - \bar{x})^2}{n} = \frac{m}{n} \sigma_{\text{before}}^2 \implies \frac{\sigma_{\text{after}}}{\sigma_{\text{before}}} = \sqrt{\frac{m}{n}} = \sqrt{1 - p}.$$

Here  $m = 6$  observed of  $n = 8$ , so the population variance is multiplied by  $6/8 = 0.75$  and the sample variance by  $(m - 1)/(n - 1) = 5/7 = 0.7143$ .

This is not an approximation

It is exact, and it is the reason mean imputation is the worst defensible thing you can do. **You have invented confidence you do not have.**

## The same thing in code

```
marks = np.array([55, 62, np.nan, 71, 58, np.nan, 90, 64])
obs = marks[~np.isnan(marks)]
for name, fill in (("mean", obs.mean()), ("median", np.median(obs))):
    f = np.where(np.isnan(marks), fill, marks)
    print(name, f.mean(), f.std(ddof=1))
```

```
mean of observed      66.6667
median of observed    63.0000
sd of observed (ddof=1) 12.6754
mean imputed: mean 66.6667 sd 10.7127 sd is now 84.5% of the truth
median imputed: mean 65.7500 sd 10.8463 sd is now 85.6% of the truth
```

$10.7127/12.6754 = 0.8452$ , and  $\sqrt{5/7} = 0.8452$ . The formula and the library agree to four decimal places, because they are the same arithmetic.

## Predict first #2

Take a real column — body mass index for the 442 diabetes patients,  $sd = 4.42 \text{ kg/m}^2$ .

Hide **40%** of it at random and fill every hole with the mean of what is left.

**What is the standard deviation now? And what happens to its correlation with the target?**

## Predict first #2

Take a real column — body mass index for the 442 diabetes patients,  $sd = 4.42 \text{ kg/m}^2$ .  
Hide **40%** of it at random and fill every hole with the mean of what is left.

**What is the standard deviation now? And what happens to its correlation with the target?**

```
p=0.1  measured sd ratio 0.9486  sqrt(1-p) = 0.9487
p=0.3  measured sd ratio 0.8363  sqrt(1-p) = 0.8367
p=0.5  measured sd ratio 0.7059  sqrt(1-p) = 0.7071
p=0.7  measured sd ratio 0.5447  sqrt(1-p) = 0.5477
complete data      corr(BMI, y) = 0.5865
40% mean-imputed   corr(BMI, y) = 0.4538 (77% of it survives)
```

## Predict first #2

Take a real column — body mass index for the 442 diabetes patients,  $sd = 4.42 \text{ kg/m}^2$ .  
Hide **40%** of it at random and fill every hole with the mean of what is left.

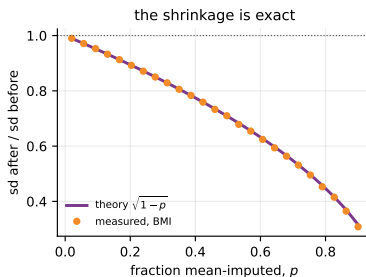
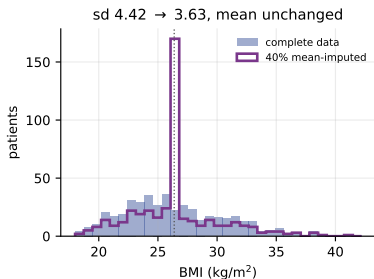
**What is the standard deviation now? And what happens to its correlation with the target?**

```
p=0.1  measured sd ratio 0.9486  sqrt(1-p) = 0.9487
p=0.3  measured sd ratio 0.8363  sqrt(1-p) = 0.8367
p=0.5  measured sd ratio 0.7059  sqrt(1-p) = 0.7071
p=0.7  measured sd ratio 0.5447  sqrt(1-p) = 0.5477
complete data      corr(BMI, y) = 0.5865
40% mean-imputed   corr(BMI, y) = 0.4538 (77% of it survives)
```

The variance shrinks by  $1 - p$ , and a quarter of the signal goes

Four values of  $p$ , 2000 repetitions each, never more than 0.0031 from  $\sqrt{1 - p}$ . **Every correlation, every regression coefficient and every  $p$ -value computed afterwards is attenuated**, and nothing in the output warns you.

## Drawn: a spike where a distribution used to be



Left: the same column before and after. Mean imputation does not blur the distribution — it **piles every hidden patient onto a single value**,  $4.4181 \rightarrow 3.6309$ . Right: the average of 400 draws per point against  $\sqrt{1-p}$ , largest disagreement 0.0080 over the whole range.

# MCAR, MAR, MNAR: the taxonomy that matters

Write  $R$  for the indicator “this value is missing”,  $X_{\text{obs}}$  for what you can see and  $X_{\text{mis}}$  for what you cannot.

**MCAR** missing completely at random    **MAR** missing at random    **MNAR** missing not at random

|             | Depends on               | Example                                                |
|-------------|--------------------------|--------------------------------------------------------|
| <b>MCAR</b> | nothing: $P(R)$ constant | a tube was dropped in the lab                          |
| <b>MAR</b>  | $X_{\text{obs}}$ only    | BMI recorded only for the high-blood-pressure patients |
| <b>MNAR</b> | $X_{\text{mis}}$ itself  | the heaviest patients decline to be weighed            |

## Why you should care

Under **MCAR** the complete cases are a fair sample — you lose power, not correctness. Under **MAR** a good imputer that uses the other columns can recover the truth. Under **MNAR** *nothing in the data can*, because the information you need was never recorded.

## The three mechanisms, measured

Same column, same 40–48% missing, three different reasons:

```
rng = np.random.default_rng(0) # the one seed for this lecture
mcar = rng.random(n) < 0.40 # a coin
mar = bp < np.median(bp) # depends on BP, which we can see
mnar = bmi > np.quantile(bmi, 0.60) # depends on BMI itself
```

|                        |               |                    |         |              |
|------------------------|---------------|--------------------|---------|--------------|
| complete-data mean BMI | 26.3758 kg/m2 | (n = 442)          |         |              |
| MCAR                   | 35.3% missing | complete-case mean | 26.4913 | bias +0.1155 |
| MAR                    | 48.0% missing | complete-case mean | 27.7230 | bias +1.3473 |
| MNAR                   | 40.0% missing | complete-case mean | 23.4253 | bias -2.9505 |

## The three mechanisms, measured

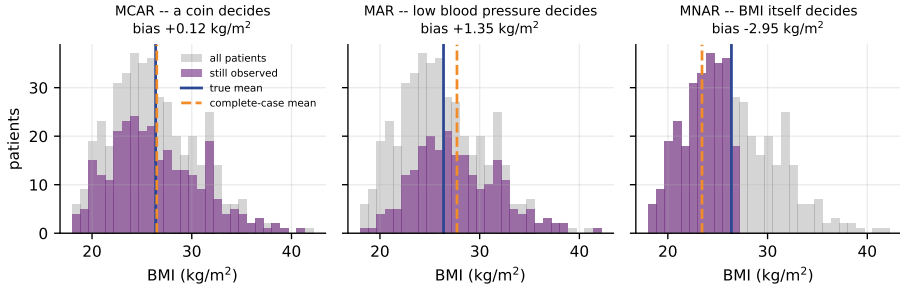
Same column, same 40–48% missing, three different reasons:

```
rng = np.random.default_rng(0) # the one seed for this lecture
mcar = rng.random(n) < 0.40 # a coin
mar = bp < np.median(bp) # depends on BP, which we can see
mnar = bmi > np.quantile(bmi, 0.60) # depends on BMI itself
```

|                        |               |                    |         |              |
|------------------------|---------------|--------------------|---------|--------------|
| complete-data mean BMI | 26.3758 kg/m2 | (n = 442)          |         |              |
| MCAR                   | 35.3% missing | complete-case mean | 26.4913 | bias +0.1155 |
| MAR                    | 48.0% missing | complete-case mean | 27.7230 | bias +1.3473 |
| MNAR                   | 40.0% missing | complete-case mean | 23.4253 | bias -2.9505 |

Throwing away the incomplete rows is *almost* free under MCAR (+0.12, and it would change sign with another draw), wrong by +1.35 under MAR, and wrong by −2.95 under MNAR — **two thirds of a standard deviation**, from a decision you never made.

# Drawn: the same histogram, three ways of losing it



Grey is everyone; purple is who is left. Under MCAR the purple bars are a shrunken copy of the grey. Under MNAR the right-hand tail is **simply gone**, and no amount of cleverness puts it back.

## Can a better imputer repair it?

IterativeImputer (MICE) predicts each column from all the others, round-robin, until it settles.

|           |              |         |      |         |
|-----------|--------------|---------|------|---------|
| MCAR mean | imputed mean | 26.4913 | bias | +0.1155 |
| MCAR MICE | imputed mean | 26.4763 | bias | +0.1005 |
| MAR mean  | imputed mean | 27.7230 | bias | +1.3473 |
| MAR MICE  | imputed mean | 26.2170 | bias | -0.1588 |
| MNAR mean | imputed mean | 23.4253 | bias | -2.9505 |
| MNAR MICE | imputed mean | 23.7924 | bias | -2.5834 |

## Can a better imputer repair it?

IterativeImputer (MICE) predicts each column from all the others, round-robin, until it settles.

|           |              |         |      |         |
|-----------|--------------|---------|------|---------|
| MCAR mean | imputed mean | 26.4913 | bias | +0.1155 |
| MCAR MICE | imputed mean | 26.4763 | bias | +0.1005 |
| MAR mean  | imputed mean | 27.7230 | bias | +1.3473 |
| MAR MICE  | imputed mean | 26.2170 | bias | -0.1588 |
| MNAR mean | imputed mean | 23.4253 | bias | -2.9505 |
| MNAR MICE | imputed mean | 23.7924 | bias | -2.5834 |

Read the middle pair, then the last pair

Under MAR, MICE cuts the bias from +1.35 to -0.16 — an 8.5× **improvement**, because blood pressure was recorded and BMI can be predicted from it. Under MNAR it moves -2.95 to -2.58 and stops. **The taxonomy is not academic: it tells you whether to bother.**

## And what it does to a coefficient you would report

Regress the target on all ten features and read off the BMI coefficient.

|                        |        |
|------------------------|--------|
| true BMI coefficient   | 5.603  |
| MNAR + mean imputation | -0.413 |
| MNAR + MICE            | 2.299  |

## And what it does to a coefficient you would report

Regress the target on all ten features and read off the BMI coefficient.

|                        |        |
|------------------------|--------|
| true BMI coefficient   | 5.603  |
| MNAR + mean imputation | -0.413 |
| MNAR + MICE            | 2.299  |

### The sign flipped

Mean-imputing an MNAR column does not merely weaken the relationship. It replaces the entire upper 40% of the distribution with one constant, and the fitted slope through what remains points the *other way*. **You would have written “BMI is unrelated to disease progression” in a report.**

## And what it does to a coefficient you would report

Regress the target on all ten features and read off the BMI coefficient.

|                        |        |
|------------------------|--------|
| true BMI coefficient   | 5.603  |
| MNAR + mean imputation | -0.413 |
| MNAR + MICE            | 2.299  |

### The sign flipped

Mean-imputing an MNAR column does not merely weaken the relationship. It replaces the entire upper 40% of the distribution with one constant, and the fitted slope through what remains points the *other way*. **You would have written “BMI is unrelated to disease progression” in a report.**

Prediction is more forgiving than inference — but only because nobody reads a predictor's coefficients.

## So which imputer should you use?

load\_diabetes, holes punched in the *training* matrix only, clean test set, 20 random splits:

|                        | 10% missing | 30% missing        |
|------------------------|-------------|--------------------|
| oracle, no missingness | 0.4762      | 0.4762             |
| drop incomplete rows   | 0.4264      | -4.6952 (+/- 7.88) |
| mean                   | 0.4721      | 0.4351             |
| median                 | 0.4727      | 0.4364             |
| constant 0             | 0.4723      | 0.4365             |
| KNN, k=5               | 0.4743      | 0.4591             |
| iterative (MICE)       | 0.4751      | 0.2639 (+/- 0.50)  |

## So which imputer should you use?

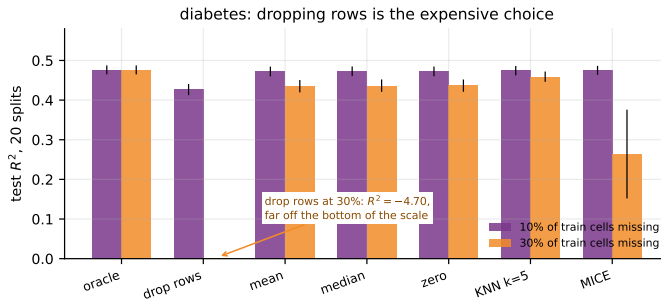
load\_diabetes, holes punched in the *training* matrix only, clean test set, 20 random splits:

|                        | 10% missing | 30% missing        |
|------------------------|-------------|--------------------|
| oracle, no missingness | 0.4762      | 0.4762             |
| drop incomplete rows   | 0.4264      | -4.6952 (+/- 7.88) |
| mean                   | 0.4721      | 0.4351             |
| median                 | 0.4727      | 0.4364             |
| constant 0             | 0.4723      | 0.4365             |
| KNN, k=5               | 0.4743      | 0.4591             |
| iterative (MICE)       | 0.4751      | 0.2639 (+/- 0.50)  |

### Deletion costs 0.05; the choice of imputer costs 0.003

At 10% every imputer lands within 0.003 of every other and within 0.004 of the oracle. **Do not agonise over the imputer. Do not drop the rows.** At 30%, KNN pulls ahead — and MICE became unstable on two of twenty splits, which is why its error bar is a factor of ten larger.

## Drawn: the same table



Purple 10%, orange 30%; error bars are standard errors over 20 splits. Six of the seven bars are the same height. The one that is not is the one everybody's first instinct reaches for.

## Missingness is itself data

A weak feature, missing 80% of the time for one class and 20% for the other — a test ordered only when the clinician already suspects something.

```
SimpleImputer(strategy="mean", add_indicator=True)
```

```
mean imputation only      0.6294
mean imputation + indicator 0.7969
difference                 +0.1675
```

## Missingness is itself data

A weak feature, missing 80% of the time for one class and 20% for the other — a test ordered only when the clinician already suspects something.

```
SimpleImputer(strategy="mean", add_indicator=True)
```

|                             |         |
|-----------------------------|---------|
| mean imputation only        | 0.6294  |
| mean imputation + indicator | 0.7969  |
| difference                  | +0.1675 |

### Sixteen points of accuracy

from a column of True/False that you get for free with one keyword argument.

### And a warning

If the reason for the hole will not exist at prediction time, that indicator is **leakage**, not a feature.

## Two numbers are not enough

A mean and a standard deviation are the first two moments. They say nothing about **shape** — two columns can match on both and look nothing alike.

$$m_k = \frac{1}{n} \sum_i (x_i - \bar{x})^k \quad \Rightarrow \quad \underbrace{g_1 = \frac{m_3}{m_2^{3/2}}}_{\text{skewness}} \quad \underbrace{b_2 = \frac{m_4}{m_2^2}}_{\text{kurtosis}}$$

### Skewness — an *odd* power

Sign survives, so one long tail fails to cancel.  
 $g_1 > 0$  tail to the right,  $g_1 < 0$  to the left, 0 balanced.

### Kurtosis — an *even* power

Sign destroyed, magnitude counts. A deviation of 4 weighs  $256\times$  one of 1: a statement about **tails**.

$$\text{Excess kurtosis} = b_2 - 3$$

A normal has  $b_2 = 3$  exactly, so subtracting puts the normal at zero. `scipy` returns the **excess** form by default; many textbooks report  $b_2$ . Say which you mean.

## By hand: five numbers

$x = (1, 2, 2, 3, 7)$ , so  $\bar{x} = 3$  and the deviations are  $(-2, -1, -1, 0, 4)$ .

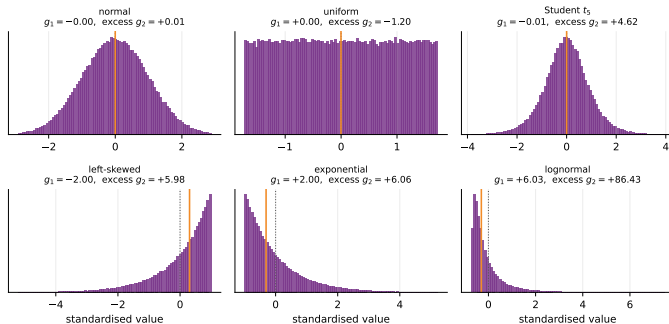
|       |                            |      |
|-------|----------------------------|------|
| $m_2$ | $(4 + 1 + 1 + 0 + 16)/5$   | 4.4  |
| $m_3$ | $(-8 - 1 - 1 + 0 + 64)/5$  | 10.8 |
| $m_4$ | $(16 + 1 + 1 + 0 + 256)/5$ | 54.8 |

In  $m_4$  the single value 7 contributes 256 of the 274 — **93% from one point in five.**

```
skewness g1 = m3 / m2^1.5 = 10.8000 / 9.2295 = +1.1702
kurtosis b2 = m4 / m2^2   = 54.8000 / 19.3600 = +2.8306
excess      = b2 - 3      = -0.1694
```

Positive skew, as the picture demands — but the excess kurtosis is **negative**, next to an obvious outlier. Hold that thought.

## Drawn: six shapes, two numbers each



The uniform is the one to remember: perfectly symmetric,  $g_1 = 0$ , yet excess  $g_2 = -1.2$  because it stops dead and has no tails at all. Negative is not a defect — it is a bounded variable.

## Predict first: what does kurtosis measure?

A normal, and a mixture: 95% drawn with sd 0.8, 5% with sd 3.0. Both standardised to sd 1. Textbooks say kurtosis is “peakedness”.

**Is the mixture's high kurtosis coming from its peak or its tails?**

## Predict first: what does kurtosis measure?

A normal, and a mixture: 95% drawn with sd 0.8, 5% with sd 3.0. Both standardised to sd 1. Textbooks say kurtosis is “peakedness”.

**Is the mixture's high kurtosis coming from its peak or its tails?**

|              | density at 0 | % $ z >3$ | excess kurt | share of sum $d^4$ |
|--------------|--------------|-----------|-------------|--------------------|
| normal       | 0.4002       | 0.2830    | 0.0079      | 0.1134             |
| 95/5 mixture | 0.4941       | 1.5005    | 9.0433      | 0.8761             |

## Predict first: what does kurtosis measure?

A normal, and a mixture: 95% drawn with sd 0.8, 5% with sd 3.0. Both standardised to sd 1. Textbooks say kurtosis is “peakedness”.

**Is the mixture’s high kurtosis coming from its peak or its tails?**

|              | density at 0 | % $ z >3$ | excess kurt | share of sum $d^4$ |
|--------------|--------------|-----------|-------------|--------------------|
| normal       | 0.4002       | 0.2830    | 0.0079      | 0.1134             |
| 95/5 mixture | 0.4941       | 1.5005    | 9.0433      | 0.8761             |

**Both are true — and the decomposition settles it**

The mixture *is* 24% taller at the centre. But split  $\sum d^4$  by where it comes from: **1.5% of the rows supply 88% of the sum**. Small deviations to the fourth power are nothing. Kurtosis is an outlier-frequency statistic.

## Both estimates are noisy, and the thresholds are folklore

“ $|g_1| > 0.5$  means skewed.” Draw 1000 samples from a *perfectly symmetric* normal and look at what they report:

| n    | mean skew | sd skew | sqrt(6/n) | sd exc k |
|------|-----------|---------|-----------|----------|
| 20   | -0.0387   | 0.4567  | 0.5477    | 0.7289   |
| 30   | -0.0223   | 0.3932  | 0.4472    | 0.6891   |
| 100  | -0.0037   | 0.2394  | 0.2449    | 0.4468   |
| 1000 | +0.0032   | 0.0787  | 0.0775    | 0.1531   |

at  $n=30$ , 20.4% of samples from a perfectly symmetric distribution report  $|\text{skewness}| > 0.5$ , the threshold often quoted as 'skewed'

$SE \approx \sqrt{6/n}$  for skewness,  $\sqrt{24/n}$  for excess kurtosis. At  $n = 30$ , **one sample in five** clears a threshold it should never reach.

## And both have a ceiling, exactly like the z-score

$$|g_1| \leq \frac{n-2}{\sqrt{n-1}}, \quad b_2 \leq \frac{n^2 - 3n + 3}{n-1}$$

| n   | one outlier | best 2-point split | $(n^2-3n+3)/(n-1) - 3$ |
|-----|-------------|--------------------|------------------------|
| 5   | 0.2500      | 0.2500             | 0.2500                 |
| 30  | 25.0345     | 25.0345            | 25.0345                |
| 569 | 564.0018    | 564.0018           | 564.0018               |

### That is why five points gave $-0.17$

At  $n = 5$  the largest excess kurtosis *any* five numbers can produce is **0.25**. Our example was most of the way to the maximum. It never said “light tails”; it said “as heavy as five points are permitted to be”. Below about fifty rows, plot the column instead.

## Why it decides the next two sections

| column       | skew   | exc k  | mean    | median  | gap % |
|--------------|--------|--------|---------|---------|-------|
| mean texture | +0.649 | +0.74  | 19.290  | 18.840  | 2.4%  |
| worst area   | +1.854 | +4.35  | 880.583 | 686.500 | 28.3% |
| area error   | +5.433 | +48.77 | 40.337  | 24.530  | 64.4% |

- **Which imputer.** At  $g_1 = 0.65$  mean and median differ by 2.4% and the choice hardly matters. At  $g_1 = 5.43$  the mean sits 64% above the median and above 71% of the rows.
- **How to read an outlier rule.**  $g_2 = 48.8$  guarantees a lot of flagged rows — and they are the shape, not contamination.
- **Whether a transform worked.** Both numbers give you a target.

## Three kinds of outlier, one word

### An error

Age = 200. A stuck sensor. A pressure in the wrong units.

**Fix or remove it.**

### A rare truth

A genuine very large tumour.

**Keep it. Use a robust method.**

### The point

Fraud. A failing machine.  
A new phenomenon.

**It is the signal.**

No statistical rule can tell these three apart

A rule tells you a point is *unusual*. Only domain knowledge tells you what to do about it. **Detection is statistics; disposal is a judgement you must be able to defend.**

Iglewicz and Hoaglin's three words: *labelling, accommodation, identification*. Today is mostly labelling.

## Three rules you will meet

|             | Statistic                                    | Flag when                     | Breakdown point                            |
|-------------|----------------------------------------------|-------------------------------|--------------------------------------------|
| z-score     | $z_i = \frac{x_i - \bar{x}}{s}$              | $ z_i  > 3$                   | 0%: one point moves both $\bar{x}$ and $s$ |
| Tukey / IQR | $IQR = Q_3 - Q_1$                            | outside $Q_{1,3} \mp 1.5 IQR$ | 25%                                        |
| modified z  | $M_i = \frac{0.6745 (x_i - \tilde{x})}{MAD}$ | $ M_i  > 3.5$                 | 50%: median and MAD                        |

### Where the two magic constants come from

$MAD = \text{median } |x_i - \tilde{x}|$ . For Normal data  $MAD \approx 0.6745 \sigma$ , so dividing by 0.6745 puts  $M$  back on the z scale. The 1.5 in Tukey's fence flags 0.70% of Normal data;  $|z| > 3$  flags 0.27%.

## By hand: thirty-five readings from one sensor

Thirty genuine readings scattered around 10.0, then the sensor sticks and reports 100.0 five times.

|                |        |                    |        |
|----------------|--------|--------------------|--------|
| mean $\bar{x}$ | 22.503 | median $\tilde{x}$ | 9.950  |
| sd $s$         | 32.109 | MAD                | 0.540  |
| $Q_1$          | 9.245  | $Q_3$              | 10.330 |

Now score one stuck reading,  $x = 100$ , with each rule:

$$z = \frac{100 - 22.503}{32.109} = 2.4135, \quad M = \frac{0.6745(100 - 9.950)}{0.540} = 112.48.$$

2.41 against a threshold of 3

The five stuck readings dragged  $\bar{x}$  from 9.59 up to 22.50 and inflated  $s$  from 0.84 to 32.11. **They are being measured against a ruler they themselves bent.**

## Predict first #3

Thirty good readings near 10. Five readings of exactly 100 — as wrong as a reading can be, and you can see them from across the room.

**How many of the five does the  $|z| > 3$  rule catch? What about the IQR fence and the modified  $z$ ?**

## Predict first #3

Thirty good readings near 10. Five readings of exactly 100 — as wrong as a reading can be, and you can see them from across the room.

**How many of the five does the  $|z| > 3$  rule catch? What about the IQR fence and the modified  $z$ ?**

| rule                  | flags | of which stuck | of which genuine |
|-----------------------|-------|----------------|------------------|
| $ z  > 3$             | 0     | 0              | 0                |
| outside 1.5*IQR fence | 6     | 5              | 1                |
| modified $z$   > 3.5  | 5     | 5              | 0                |

sd WITHOUT the five stuck readings 0.837  
z of a stuck reading would then be 108.1

## Predict first #3

Thirty good readings near 10. Five readings of exactly 100 — as wrong as a reading can be, and you can see them from across the room.

**How many of the five does the  $|z| > 3$  rule catch? What about the IQR fence and the modified  $z$ ?**

| rule                  | flags | of which stuck | of which genuine |
|-----------------------|-------|----------------|------------------|
| $ z  > 3$             | 0     | 0              | 0                |
| outside 1.5*IQR fence | 6     | 5              | 1                |
| modified $z$ $> 3.5$  | 5     | 5              | 0                |

sd WITHOUT the five stuck readings 0.837  
z of a stuck reading would then be 108.1

**Zero.** The most-taught rule in the subject finds none of them

And it is not close: 2.41 against 3. Remove the outliers first and the very same reading scores  $z = 108$ .

**This is masking: an outlier inflates the standard deviation it is being divided by, and hides inside its own damage.**

## The same thing in code

```
z = (v - v.mean()) / v.std(ddof=1)
q1, q3 = np.percentile(v, [25, 75]); iqr = q3 - q1
mad = np.median(np.abs(v - np.median(v)))
mz = 0.6745 * (v - np.median(v)) / mad
```

```
mean 22.503   sd 32.109   median 9.950   MAD 0.540
Q1 9.245   Q3 10.330   IQR 1.085   upper fence Q3+1.5*IQR 11.957
a stuck reading: z = 2.4135   (needs > 3)   modified z = 112.48   (needs > 3.5)
```

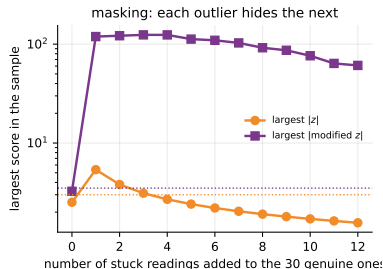
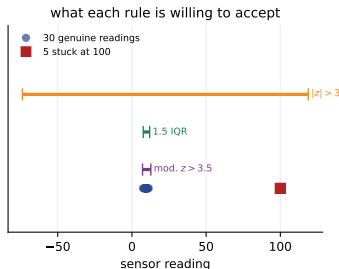
## The same thing in code

```
z = (v - v.mean()) / v.std(ddof=1)
q1, q3 = np.percentile(v, [25, 75]); iqr = q3 - q1
mad = np.median(np.abs(v - np.median(v)))
mz = 0.6745 * (v - np.median(v)) / mad
```

```
mean 22.503   sd 32.109   median 9.950   MAD 0.540
Q1 9.245   Q3 10.330   IQR 1.085   upper fence Q3+1.5*IQR 11.957
a stuck reading: z = 2.4135   (needs > 3)   modified z = 112.48   (needs > 3.5)
```

Four lines each. The whole difference is that the middle rule uses **quartiles** and the third uses **a median and a median** — and a median does not care how far away the far-away points are.

## Drawn: masking, and why it gets worse



Left: what each rule is willing to accept. The  $z$  interval reaches from  $-74$  to  $+119$ ; the other two are slivers around 10. Right: add stuck readings one at a time. **One outlier is caught at  $z = 5.38$ , two at  $3.81$ , three at  $3.11$ , and from the fourth onwards none ever again.** The modified  $z$  stays above 60 throughout.

## By hand: the rule has a ceiling

For a sample of  $n$  values with sample standard deviation  $s$  (the  $n - 1$  one), no observation can have

$$|z| > \frac{n-1}{\sqrt{n}}.$$

| n  | largest possible $ z $ | can the rule $ z  > 3$ ever fire? |
|----|------------------------|-----------------------------------|
| 3  | 1.1547                 | NO -- not ever                    |
| 5  | 1.7889                 | NO -- not ever                    |
| 10 | 2.8460                 | NO -- not ever                    |
| 11 | 3.0151                 | yes                               |
| 50 | 6.9296                 | yes                               |

## By hand: the rule has a ceiling

For a sample of  $n$  values with sample standard deviation  $s$  (the  $n - 1$  one), no observation can have

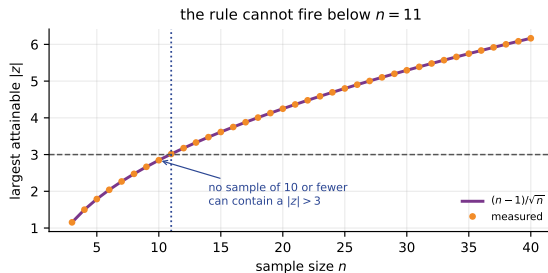
$$|z| > \frac{n-1}{\sqrt{n}}.$$

| n  | largest possible $ z $ | can the rule $ z  > 3$ ever fire? |
|----|------------------------|-----------------------------------|
| 3  | 1.1547                 | NO -- not ever                    |
| 5  | 1.7889                 | NO -- not ever                    |
| 10 | 2.8460                 | NO -- not ever                    |
| 11 | 3.0151                 | yes                               |
| 50 | 6.9296                 | yes                               |

Below  $n = 11$  the rule is not strict — it is inert

Whatever the data, whatever the contamination. If you have ten measurements and you apply  $|z| > 3$ , you have written a line of code that can only ever return “no outliers”. NIST states the bound in exactly these words, and the fix in the next line: use the modified  $z$ .

## Drawn: the ceiling



Theory and measurement agree to  $9 \times 10^{-16}$ . The curve crosses 3 between  $n = 10$  (2.846) and  $n = 11$  (3.015). **The threshold that everybody memorises has a domain of validity that nobody mentions.**

## The opposite failure: a skewed column

The most skewed column of `load_breast_cancer` — 'area error', skewness 5.43, no contamination at all.

| rule                               | flags | % of rows |
|------------------------------------|-------|-----------|
| <code> z  &gt; 3</code>            | 6     | 1.1%      |
| 1.5*IQR fences                     | 65    | 11.4%     |
| <code> modified z  &gt; 3.5</code> | 81    | 14.2%     |

under a perfect Normal these rules flag 0.27%, 0.70% and 0.35% of rows

## The opposite failure: a skewed column

The most skewed column of load\_breast\_cancer — 'area error', skewness 5.43, no contamination at all.

| rule              | flags | % of rows |
|-------------------|-------|-----------|
| z  > 3            | 6     | 1.1%      |
| 1.5*IQR fences    | 65    | 11.4%     |
| modified z  > 3.5 | 81    | 14.2%     |

under a perfect Normal these rules flag 0.27%, 0.70% and 0.35% of rows

now Yeo-Johnson the column first (skewness 5.433 -> 0.069) and repeat

|                   |   |      |
|-------------------|---|------|
| z  > 3            | 0 | 0.0% |
| 1.5*IQR fences    | 1 | 0.2% |
| modified z  > 3.5 | 0 | 0.0% |

## The opposite failure: a skewed column

The most skewed column of `load_breast_cancer` — 'area error', skewness 5.43, no contamination at all.

| rule                               | flags | % of rows |
|------------------------------------|-------|-----------|
| <code> z  &gt; 3</code>            | 6     | 1.1%      |
| 1.5*IQR fences                     | 65    | 11.4%     |
| <code> modified z  &gt; 3.5</code> | 81    | 14.2%     |

under a perfect Normal these rules flag 0.27%, 0.70% and 0.35% of rows

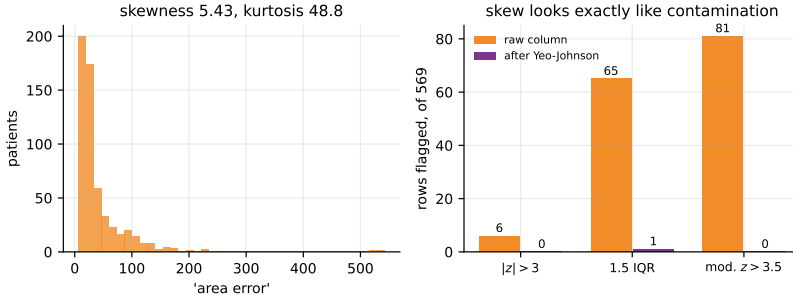
now Yeo-Johnson the column first (skewness 5.433 → 0.069) and repeat

|                                    |   |      |
|------------------------------------|---|------|
| <code> z  &gt; 3</code>            | 0 | 0.0% |
| 1.5*IQR fences                     | 1 | 0.2% |
| <code> modified z  &gt; 3.5</code> | 0 | 0.0% |

### Eighty-one “outliers” that were never outliers

Every one of these rules assumes symmetry. On a long right tail they flag the tail. **Transform first, then look for outliers** — and that is the whole argument for the next section.

## Drawn: skew looks exactly like contamination



Left: the raw column, kurtosis 48.8. Right: how many rows each rule condemns before and after a single power transform.  $6 \rightarrow 0$ ,  $65 \rightarrow 1$ ,  $81 \rightarrow 0$  — and not one data value was deleted.

# Numerical methods, 1: binning and smoothing

Nine sorted prices, three equal-depth bins. This is Han, Kamber and Pei's standard treatment of *noisy* data, and it is a favourite exam question.

| Bin | Values     | mean | median | boundaries |
|-----|------------|------|--------|------------|
| 1   | 4, 8, 15   | 9    | 8      | 4 and 15   |
| 2   | 21, 21, 24 | 22   | 21     | 21 and 24  |
| 3   | 25, 28, 34 | 29   | 28     | 25 and 34  |

|                          |                                  |
|--------------------------|----------------------------------|
| by bin <b>means</b>      | 9, 9, 9, 22, 22, 22, 29, 29, 29  |
| by bin <b>medians</b>    | 8, 8, 8, 21, 21, 21, 28, 28, 28  |
| by bin <b>boundaries</b> | 4, 4, 15, 21, 21, 24, 25, 25, 34 |

## Read the third row

Each value is replaced by whichever *end* of its bin is closer: 8 is nearer 4 than 15, so it becomes 4; 15 stays 15.

## Numerical methods, 2: five things to do to an outlier

Back to the thirty-five sensor readings. The truth is mean 9.586, sd 0.837.

|                         |      |        |    |        |   |    |
|-------------------------|------|--------|----|--------|---|----|
| leave them alone        | mean | 22.503 | sd | 32.109 | n | 35 |
| winsorise at 5 and 95   | mean | 22.523 | sd | 32.100 | n | 35 |
| winsorise at 15 and 85  | mean | 9.903  | sd | 0.839  | n | 35 |
| cap at the IQR fence    | mean | 9.925  | sd | 1.143  | n | 35 |
| drop by modified z      | mean | 9.586  | sd | 0.837  | n | 30 |
| the 30 genuine readings | mean | 9.586  | sd | 0.837  | n | 30 |

## Numerical methods, 2: five things to do to an outlier

Back to the thirty-five sensor readings. The truth is mean 9.586, sd 0.837.

|                         |      |        |    |        |   |    |
|-------------------------|------|--------|----|--------|---|----|
| leave them alone        | mean | 22.503 | sd | 32.109 | n | 35 |
| winsorise at 5 and 95   | mean | 22.523 | sd | 32.100 | n | 35 |
| winsorise at 15 and 85  | mean | 9.903  | sd | 0.839  | n | 35 |
| cap at the IQR fence    | mean | 9.925  | sd | 1.143  | n | 35 |
| drop by modified z      | mean | 9.586  | sd | 0.837  | n | 30 |
| the 30 genuine readings | mean | 9.586  | sd | 0.837  | n | 30 |

### Look at the second row before anything else

Winsorising at the 5th and 95th percentiles did **nothing**, because  $5/35 = 14\%$  of the data is contaminated and you told it to clip 10%. **A fixed percentile is a guess about how much dirt there is.** The IQR fence and the modified z work it out from the data.

## And the scaler you choose is an outlier decision

```
StandardScaler().fit_transform(sensor.reshape(-1, 1)) # (x - mean) / sd  
RobustScaler().fit_transform(sensor.reshape(-1, 1)) # (x - median) / IQR
```

```
StandardScaler  30 genuine readings span -0.4747 to -0.3527 (width 0.1220)  
RobustScaler    30 genuine readings span -2.2765 to  1.2811 (width 3.5576)
```

## And the scaler you choose is an outlier decision

```
StandardScaler().fit_transform(sensor.reshape(-1, 1)) # (x - mean) / sd  
RobustScaler().fit_transform(sensor.reshape(-1, 1)) # (x - median) / IQR
```

```
StandardScaler  30 genuine readings span -0.4747 to -0.3527 (width 0.1220)  
RobustScaler    30 genuine readings span -2.2765 to  1.2811 (width 3.5576)
```

**The same thirty readings, squashed into 0.12 of a unit or spread over 3.56.**  
StandardScaler spent its whole dynamic range accommodating five wrong numbers, and every distance-based model downstream now thinks those thirty patients are identical.

# Why transform a variable at all

## Four honest reasons

1. Symmetry: a mean and an sd stop lying.
2. Linearity: a curved relationship straightens.
3. Equal spread: residual variance stops growing with the fit.
4. Outlier rules start working (previous section).

## The ladder of powers

...,  $-1/x$ ,  $-1/\sqrt{x}$ ,  $\log x$ ,  $\sqrt{x}$ ,  $x$ ,  $x^2$ , ...

Move **down** to pull in a right tail, **up** for a left one.  $\log$  sits in the middle and is the one you will use most.

## Two things a transform is not

It is not scaling (that is L6, and it does not change shape). And it is not free: after  $\log$ , a coefficient is a statement about *percentage* change, and you must say so.

## Four transformations, one real column

'area error': 569 values, minimum 6.80, maximum 542.2.

| transformation      | skewness | kurtosis |
|---------------------|----------|----------|
| none                | 5.4328   | 48.7672  |
| sqrt(x)             | 2.1363   | 7.8193   |
| log(x)              | 0.7955   | 0.3622   |
| 1/sqrt(x)           | 0.0447   | -0.3685  |
| Box-Cox, lam=-0.436 | 0.0584   | -0.4094  |
| Yeo-Johnson         | 0.0691   | -0.4561  |

## Four transformations, one real column

'area error': 569 values, minimum 6.80, maximum 542.2.

| transformation      | skewness | kurtosis |
|---------------------|----------|----------|
| none                | 5.4328   | 48.7672  |
| sqrt(x)             | 2.1363   | 7.8193   |
| log(x)              | 0.7955   | 0.3622   |
| 1/sqrt(x)           | 0.0447   | -0.3685  |
| Box-Cox, lam=-0.436 | 0.0584   | -0.4094  |
| Yeo-Johnson         | 0.0691   | -0.4561  |

Skewness 5.43  $\rightarrow$  0.06, kurtosis 48.8  $\rightarrow$  -0.41. The square root was not enough; the log almost was; the reciprocal square root went slightly past.

## Four transformations, one real column

'area error': 569 values, minimum 6.80, maximum 542.2.

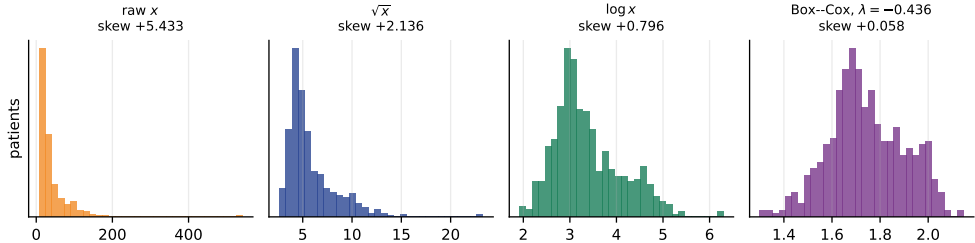
| transformation      | skewness | kurtosis |
|---------------------|----------|----------|
| none                | 5.4328   | 48.7672  |
| sqrt(x)             | 2.1363   | 7.8193   |
| log(x)              | 0.7955   | 0.3622   |
| 1/sqrt(x)           | 0.0447   | -0.3685  |
| Box-Cox, lam=-0.436 | 0.0584   | -0.4094  |
| Yeo-Johnson         | 0.0691   | -0.4561  |

Skewness 5.43  $\rightarrow$  0.06, kurtosis 48.8  $\rightarrow$  -0.41. The square root was not enough; the log almost was; the reciprocal square root went slightly past.

The ladder is discrete, and the right rung is between two of them

That is precisely the gap Box-Cox fills.

## Drawn: the same 569 numbers, four ways



Nothing has been deleted and no value has changed its rank. All that changed is the ruler the values are printed on — and the fourth panel is a variable you can put a mean and a standard deviation on.

## Box–Cox and Yeo–Johnson

$$x^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \log x, & \lambda = 0 \end{cases} \quad x > 0 \text{ required}$$

### Why the $-1$ and the $/\lambda$

So that the family is *continuous* at  $\lambda = 0$ :  $(x^\lambda - 1)/\lambda \rightarrow \log x$ . Without them,  $\lambda \rightarrow 0$  would give the constant 1.

### Yeo–Johnson

The same idea rewritten so that it works for  $x \leq 0$ : it applies the Box–Cox form to  $x + 1$  on the positive side and a mirrored version to  $-x + 1$  on the negative side.

$\lambda$  is chosen by **maximum likelihood** under a Normal model, not by minimising skewness — which is why the two answers differ slightly.

## By hand: the family on four numbers

$x = (1, 4, 16, 64)$  — a *geometric* sequence, each value  $4 \times$  the last.

```
lambda +1.00 -> [ 0.  3. 15. 63.]
lambda +0.50 -> [ 0.  2.  6. 14.]
lambda +0.00 -> [0.    1.3863 2.7726 4.1589]
lambda -0.50 -> [-0.    1.    1.5   1.75]
lambda -1.00 -> [-0.    0.75  0.9375 0.9844]
```

Check one with a pen:  $\lambda = -0.5$ ,  $x = 16$ .  $16^{-0.5} = 1/4 = 0.25$ , so  $(0.25 - 1)/(-0.5) = \mathbf{1.5}$  — the third entry.

### Read the $\lambda = 0$ row

0, 1.3863, 2.7726, 4.1589 — the gaps are all  $\log 4$ . The log turned a **geometric** sequence into an **arithmetic** one. That is what “the log is the right ruler for multiplicative data” actually means.

## Yeo–Johnson: the same idea on the whole real line

$$y^{(\lambda)} = \begin{cases} \frac{(y+1)^\lambda - 1}{\lambda}, & \lambda \neq 0, y \geq 0 \\ \log(y+1), & \lambda = 0, y \geq 0 \\ -\frac{(-y+1)^{2-\lambda} - 1}{2-\lambda}, & \lambda \neq 2, y < 0 \\ -\log(-y+1), & \lambda = 2, y < 0 \end{cases}$$

Four cases, one idea: the lower pair is the upper pair **reflected** — negate the input, swap  $\lambda$  for  $2 - \lambda$ , negate the result.

```
lambda 0.0 -> [-7.5   -1.5   -0.625  0.      0.4055  0.6931  1.3863]
lambda 1.0 -> [-3.    -1.    -0.5    0.     0.5    1.     3.   ]
lambda 2.0 -> [-1.3863 -0.6931 -0.4055  0.      0.625   1.5    7.5   ]

our implementation vs sklearn on 'area error', lambda = -0.483636
max |difference| = 0.000e+00
```

$\lambda = 1$  is the identity, every  $\lambda$  maps  $0 \rightarrow 0$ , and the  $\lambda = 0$  and  $\lambda = 2$  rows are mirror images.

## Predict first: what picks $\lambda$ ?

Maximise the Normal log-likelihood of the *transformed* column:

$$\ell(\lambda) = -\frac{n}{2} \log \hat{\sigma}_\lambda^2 + \underbrace{(\lambda - 1) \sum_i \log x_i}_{\text{Jacobian}}$$

**Every summary drops the second term. What happens if you do?**

## Predict first: what picks $\lambda$ ?

Maximise the Normal log-likelihood of the *transformed* column:

$$\ell(\lambda) = -\frac{n}{2} \log \hat{\sigma}_\lambda^2 + \underbrace{(\lambda - 1) \sum_i \log x_i}_{\text{Jacobian}}$$

**Every summary drops the second term. What happens if you do?**

```
with the Jacobian term : lambda* = -0.4360
scipy.stats.boxcox     : lambda* = -0.4357
without the Jacobian term : lambda* = -2.0000 <-- runs to the edge

lambda -2.000    var 2.089073e-06
lambda -0.436   var 2.458963e-02
```

## Predict first: what picks $\lambda$ ?

Maximise the Normal log-likelihood of the *transformed* column:

$$\ell(\lambda) = -\frac{n}{2} \log \hat{\sigma}_\lambda^2 + \underbrace{(\lambda - 1) \sum_i \log x_i}_{\text{Jacobian}}$$

**Every summary drops the second term. What happens if you do?**

```
with the Jacobian term : lambda* = -0.4360
scipy.stats.boxcox      : lambda* = -0.4357
without the Jacobian term : lambda* = -2.0000 <-- runs to the edge

lambda -2.000    var 2.089073e-06
lambda -0.436    var 2.458963e-02
```

**The first term alone rewards crushing the data to a point**

Smaller variance always scores better, so the optimiser runs to the edge of the grid. The Jacobian is the change-of-variables factor, and it charges for that squashing at exactly the right rate.

## Two details that confuse everybody

```
standardize=False  lambda -0.4836  mean  +1.6509  sd  0.1258  skew +0.0691
standardize=True   lambda -0.4836  mean  -0.0000  sd  1.0000  skew +0.0691

lambda fitted on train only : -0.5527
lambda fitted on all rows   : -0.4836  (this one has seen the test set)
lambda fitted on test only  : -0.3386
```

### standardize=True is the default

It transforms *and* z-scores. Skewness is identical either way — standardising is linear and cannot change shape.

### $\lambda$ is a *learned parameter*

Three splits, three values. Fit it on all the rows and you have leaked, exactly as with a scaler or an imputer. It goes in the Pipeline.

## $\lambda$ chooses a rung of the ladder

```
lambda -1.00  1/x (reciprocal) skewness -0.8635
lambda -0.50  1/sqrt(x)       skewness -0.0447
lambda +0.00  log(x)          skewness +0.7955
lambda +0.50  sqrt(x)         skewness +2.1363
lambda +1.00  x (no change)   skewness +5.4328
maximum-likelihood lambda from scipy -0.4357
skewness there                                +0.0584
```

## $\lambda$ chooses a rung of the ladder

```
lambda -1.00    1/x (reciprocal)    skewness -0.8635
lambda -0.50    1/sqrt(x)          skewness -0.0447
lambda +0.00    log(x)              skewness +0.7955
lambda +0.50    sqrt(x)             skewness +2.1363
lambda +1.00    x (no change)       skewness +5.4328
maximum-likelihood lambda from scipy -0.4357
skewness there                                     +0.0584
```

```
a column that goes negative cannot use Box-Cox
diabetes 'bp': min -0.1124 max 0.1320 skewness +0.2897
stats.boxcox(bp) -> ValueError: Data must be positive.
Yeo-Johnson handles it: skewness +0.2897 -> +0.0226
```

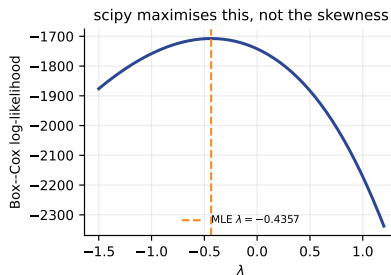
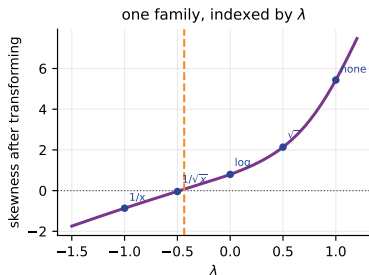
## $\lambda$ chooses a rung of the ladder

```
lambda -1.00  1/x (reciprocal)  skewness -0.8635
lambda -0.50  1/sqrt(x)         skewness -0.0447
lambda +0.00  log(x)            skewness +0.7955
lambda +0.50  sqrt(x)           skewness +2.1363
lambda +1.00  x (no change)     skewness +5.4328
maximum-likelihood lambda from scipy -0.4357
skewness there                                +0.0584
```

```
a column that goes negative cannot use Box-Cox
diabetes 'bp': min -0.1124 max 0.1320 skewness +0.2897
stats.boxcox(bp) -> ValueError: Data must be positive.
Yeo-Johnson handles it: skewness +0.2897 -> +0.0226
```

Every rung you know is a value of  $\lambda$ , and  $\lambda$  is allowed to be  $-0.4357$ .

## Drawn: what scipy is actually maximising



Left: skewness against  $\lambda$ , with the five named rungs marked. Right: the Box-Cox log-likelihood, whose peak is at  $\lambda = -0.4357$ . The  $\lambda$  that would zero the skewness exactly is  $-0.4688$  — close, but they are answers to different questions.

## Does it help the model? It depends which model

20 random splits of load\_breast\_cancer:

|                    |            |        |                   |        |         |         |
|--------------------|------------|--------|-------------------|--------|---------|---------|
| GaussianNB         | scale only | 0.9304 | Yeo-Johnson first | 0.9462 | +0.0157 | (16/20) |
| LogisticRegression | scale only | 0.9762 | Yeo-Johnson first | 0.9766 | +0.0003 | (5/20)  |

## Does it help the model? It depends which model

20 random splits of load\_breast\_cancer:

|                    |            |        |                   |        |         |         |
|--------------------|------------|--------|-------------------|--------|---------|---------|
| GaussianNB         | scale only | 0.9304 | Yeo-Johnson first | 0.9462 | +0.0157 | (16/20) |
| LogisticRegression | scale only | 0.9762 | Yeo-Johnson first | 0.9766 | +0.0003 | (5/20)  |

### Naive Bayes gains 1.6 points

It *assumes* each feature is Normal within a class. Making that assumption true is worth real accuracy, on 16 of the 20 splits.

### Logistic regression gains nothing

It makes no distributional assumption about  $X$  at all. +0.0003 is **noise**, and it **won 5 splits of 20**.

## Does it help the model? It depends which model

20 random splits of load\_breast\_cancer:

|                    |            |        |                   |        |         |         |
|--------------------|------------|--------|-------------------|--------|---------|---------|
| GaussianNB         | scale only | 0.9304 | Yeo-Johnson first | 0.9462 | +0.0157 | (16/20) |
| LogisticRegression | scale only | 0.9762 | Yeo-Johnson first | 0.9766 | +0.0003 | (5/20)  |

### Naive Bayes gains 1.6 points

It *assumes* each feature is Normal within a class. Making that assumption true is worth real accuracy, on 16 of the 20 splits.

### Logistic regression gains nothing

It makes no distributional assumption about  $X$  at all. +0.0003 is **noise**, and it **won 5 splits of 20**.

**Transform because the model assumes it, or because you need to read the variable — not as a ritual.**

# The single rule that governs all of it

Every number used to clean the data must be computed from the **training rows only**.

| Step            | The number it learns                             |
|-----------------|--------------------------------------------------|
| imputation      | the column mean, median, mode, or a fitted model |
| outlier removal | a threshold, or <i>which rows to keep</i>        |
| transformation  | $\lambda$                                        |
| scaling         | the mean and the standard deviation              |

## Break it and your test score is a measurement of nothing

The score you report is no longer an estimate of performance on unseen data, because the data was not unseen. **How much you overstate by depends entirely on which step you got wrong** — and the range is enormous.

## Predict first #4

The most human thing in data science: fit a model, look at the rows it gets badly wrong, decide they are outliers, delete them, and start again.

Concretely: fit on all 442 diabetes patients, delete the 10% with the largest residual, *then* split into train and test and report  $R^2$ .

**By how much does that overstate the honest number?**

## Predict first #4

The most human thing in data science: fit a model, look at the rows it gets badly wrong, decide they are outliers, delete them, and start again.

Concretely: fit on all 442 diabetes patients, delete the 10% with the largest residual, *then* split into train and test and report  $R^2$ .

**By how much does that overstate the honest number?**

```
LEAK C -- drop the rows your model cannot fit, THEN split. 200 splits
drop worst 5% leaky 0.5850 honest 0.4730 gap +0.1120 t +19.2 win 181/200
drop worst 10% leaky 0.6390 honest 0.4688 gap +0.1702 t +29.9 win 192/200
drop worst 20% leaky 0.7022 honest 0.4732 gap +0.2291 t +45.0 win 200/200
```

## Predict first #4

The most human thing in data science: fit a model, look at the rows it gets badly wrong, decide they are outliers, delete them, and start again.

Concretely: fit on all 442 diabetes patients, delete the 10% with the largest residual, *then* split into train and test and report  $R^2$ .

**By how much does that overstate the honest number?**

```
LEAK C -- drop the rows your model cannot fit, THEN split. 200 splits
drop worst 5% leaky 0.5850 honest 0.4730 gap +0.1120 t +19.2 win 181/200
drop worst 10% leaky 0.6390 honest 0.4688 gap +0.1702 t +29.9 win 192/200
drop worst 20% leaky 0.7022 honest 0.4732 gap +0.2291 t +45.0 win 200/200
```

0.47 became 0.64, on 192 splits out of 200

The honest column barely moves — cleaning the *training* set is fine. What moved is the test set: you removed exactly the patients your model was going to fail on. **You did not improve the model. You marked your own exam paper.**

## Three sins, all measured, and they are not equal

```
LEAK A -- fit the scaler on everything, then split. 300 splits each
  n_train = 20 leaky 0.9338 honest 0.9304 gap +0.0034 t +6.0 win 172/300
  n_train = 60 leaky 0.9587 honest 0.9577 gap +0.0011 t +4.3 win 146/300
  n_train = 427 leaky 0.9774 honest 0.9772 gap +0.0002 t +1.7 win 18/300
LEAK B -- fit the imputer on everything. 30% missing, 60 splits
  mean leaky 0.9586 honest 0.9583 gap +0.0003 t +0.8 win 9/60
  KNN k=5 leaky 0.9535 honest 0.9552 gap -0.0017 t -1.4 win 18/60
```

## Three sins, all measured, and they are not equal

```
LEAK A -- fit the scaler on everything, then split. 300 splits each
  n_train = 20 leaky 0.9338 honest 0.9304 gap +0.0034 t +6.0 win 172/300
  n_train = 60 leaky 0.9587 honest 0.9577 gap +0.0011 t +4.3 win 146/300
  n_train = 427 leaky 0.9774 honest 0.9772 gap +0.0002 t +1.7 win 18/300
LEAK B -- fit the imputer on everything. 30% missing, 60 splits
  mean leaky 0.9586 honest 0.9583 gap +0.0003 t +0.8 win 9/60
  KNN k=5 leaky 0.9535 honest 0.9552 gap -0.0017 t -1.4 win 18/60
```

### Be honest about the size of each one

Leak A is **real** —  $t = 6.0$  over 300 splits is not noise — and it is **tiny**, 0.3 of a percentage point, shrinking to nothing as the training set grows. Leak B did not show a gap at all here. Leak C was worth 0.17 of  $R^2$ .

## Three sins, all measured, and they are not equal

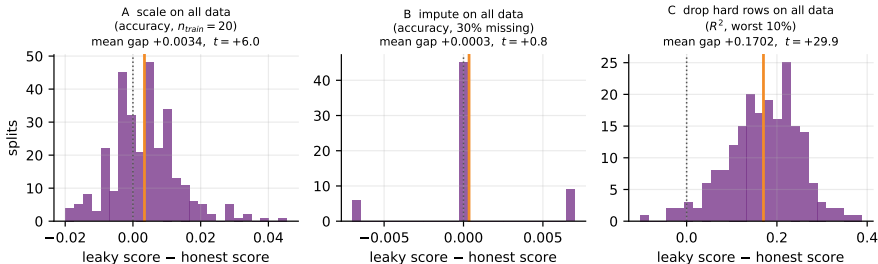
```
LEAK A -- fit the scaler on everything, then split. 300 splits each
  n_train = 20 leaky 0.9338 honest 0.9304 gap +0.0034 t +6.0 win 172/300
  n_train = 60 leaky 0.9587 honest 0.9577 gap +0.0011 t +4.3 win 146/300
  n_train = 427 leaky 0.9774 honest 0.9772 gap +0.0002 t +1.7 win 18/300
LEAK B -- fit the imputer on everything. 30% missing, 60 splits
  mean leaky 0.9586 honest 0.9583 gap +0.0003 t +0.8 win 9/60
  KNN k=5 leaky 0.9535 honest 0.9552 gap -0.0017 t -1.4 win 18/60
```

### Be honest about the size of each one

Leak A is **real** —  $t = 6.0$  over 300 splits is not noise — and it is **tiny**, 0.3 of a percentage point, shrinking to nothing as the training set grows. Leak B did not show a gap at all here. Leak C was worth 0.17 of  $R^2$ .

**The size of a leak is the amount of test information the step absorbs.** A mean absorbs almost none. A decision about which rows to keep absorbs a great deal.

## Drawn: the three gaps on the same axis idea



Each histogram is the per-split difference, leaky minus honest. Note the horizontal scales: 0.04, 0.008 and 0.4. **Do not learn “all leakage is catastrophic” and do not learn “leakage is overblown”.** Learn to ask what the step learned from the test set.

## The pipeline that cannot leak

```
pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median", add_indicator=True)),
    ("power", PowerTransformer(method="yeo-johnson")),
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=5000)),
])
cross_val_score(pipe, X_with_holes, y, cv=5).mean()
```

```
one Pipeline, 5-fold cross-validation, 10% of cells missing
steps: impute -> power -> scale -> model
cross_val_score 0.9578 +/- 0.0140
```

## The pipeline that cannot leak

```
pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median", add_indicator=True)),
    ("power", PowerTransformer(method="yeo-johnson")),
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=5000)),
])
cross_val_score(pipe, X_with_holes, y, cv=5).mean()
```

```
one Pipeline, 5-fold cross-validation, 10% of cells missing
steps: impute -> power -> scale -> model
cross_val_score 0.9578 +/- 0.0140
```

Every step is refitted inside every fold. The median,  $\lambda$ , the mean and the sd are all recomputed nine times, from four fifths of the data, and the held-out fifth is never touched. **There is nothing left for you to get wrong by hand — which is the entire reason Pipeline exists.**

## Summary — data cleaning in twelve lines

1. Deletion is exponential in the number of columns. 5% of cells missing across 30 features left 114 of 569 patients.
2. Mean imputation preserves the mean and multiplies the variance by exactly  $m/n$ : measured 0.9486 against  $\sqrt{0.9} = 0.9487$ , and 0.5447 against  $\sqrt{0.3} = 0.5477$ .
3. That shrinkage propagates:  $\text{corr}(\text{BMI}, y)$  fell from 0.5865 to 0.4538 at 40% imputed.
4. **MCAR** bias +0.12, **MAR** +1.35, **MNAR** -2.95 kg/m<sup>2</sup>. MICE fixed MAR (+1.35  $\rightarrow$  -0.16) and could not fix MNAR (-2.95  $\rightarrow$  -2.58).
5. Under MNAR, mean imputation flipped a coefficient's sign: 5.60  $\rightarrow$  -0.41.
6. Dropping incomplete rows cost 0.05 of  $R^2$ ; the choice among six imputers cost 0.003. **Spend your attention accordingly.**
7. `add_indicator=True` was worth +0.1675 accuracy when the missingness itself carried the signal.
8. **Thirty good readings and five stuck at 100: the  $|z| > 3$  rule flagged none of them.** The modified  $z$  flagged exactly five.
9. Masking gets worse with more contamination: caught at  $z = 5.38$  with one outlier, 3.11 with three, never from four onwards.
10.  $|z|$  can never exceed  $(n-1)/\sqrt{n}$ , so below  $n = 11$  the rule cannot fire at all.
11. On a skewed column the three rules flagged 6, 65 and 81 rows; after one Yeo-Johnson, 0, 1 and 0. Box-Cox took skewness 5.43  $\rightarrow$  0.06 at  $\lambda = -0.4357$ .
12. Scaling before the split was worth +0.0034; deleting badly-fitted rows before the split was worth +0.1702. **Use a Pipeline and neither can happen.**

## Before the next lecture

### Read

notes.pdf — the full variance-shrinkage derivation, the  $(n-1)/\sqrt{n}$  proof, and **10 practice questions with worked solutions**.

### Do

Questions 2, 5 and 9. Question 5 is the three outlier rules by hand; question 9 is a debugging story you will meet for real.

### Next: feature engineering

Scaling, encoding, and reducing the number of columns. Carry one thing across: **every one of those steps also learns a number from the training set**.

### And a habit to start now

Before you fit anything: how many holes, why, and what does the histogram of each column look like?

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.